



INSTITUTO POLITECNICO NACIONAL



ESCUELA SUPERIOR DE CÓMPUTO

ASIGNATURA: Aplicaciones para comunicaciones en red

EJERCICIO 3: Análisis de programas de sockets

DOCENTE: Tanibet Pérez De Los Santos

GRUPO: 6CV1

INTEGRANTES:

Alarcón Olalde Valeria Regina

Gil Valentín Oswaldo

Olguín Hernández Johan Gael

Perez Cuahutencos Andres

Tiburcio Urdiera Alexis Javier

FECHA DE ENTREGA: 17/Febrero/2026

EJERCICIO 3 - ANÁLISIS DE PROGRAMAS CON SOCKETS

Estructura de Programas con Sockets (TCP vs UDP)

Objetivo

Analizar la **estructura interna de programas que utilizan sockets**, identificando las fases del flujo de ejecución y las diferencias arquitectónicas entre TCP y UDP.

Al finalizar la práctica, el estudiante será capaz de:

- Identificar dónde se crea un socket dentro del código.
- Reconocer las fases estructurales de un servidor.
- Explicar las diferencias estructurales entre TCP y UDP.
- Inferir qué líneas faltan en un programa incompleto.

Parte 1 – Análisis estructural (TCP)

Analiza el programa echo_server de la carpeta TCP (repositorio de git)

Actividades

1. Identifique la línea donde:
 - Se crea el socket.

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as TCPServerSocket:
```

Imagen 1.

En la imagen 1, se muestra la línea de código número siete en donde se muestra que se instancia un objeto que utilizará IPV4 (AF_INET) y se indica que se usará el protocolo TCP (SOCK_STREAM).

- Se configura el puerto.

```
8 TCPServerSocket.bind((HOST, PORT))
```

Imagen 2.

En la imagen 2, se muestra la línea de código número ocho en donde se utiliza el método bind, cuyo propósito es asociar el socket a la dirección IP y al puerto especificados en las variables.

- Se inicia la espera de conexiones.

```
9 TCPServerSocket.listen()
```

Imagen 3.

En la imagen 3, se muestra la línea de código nueve, en donde se usa el método listen para escuchar las peticiones de conexión hacia el servidor.

- Se establece la comunicación.

```
12 Client_conn, Client_addr = TCPServerSocket.accept()  
13 with Client_conn:
```

Imagen 4.

En la imagen 4, tenemos la línea de código doce, en donde el servidor acepta la conexión entrante.

- Se cierran los recursos.

```

13  ✓      with Client_conn:
14          print("Conectado a", Client_addr)
15  ✓      while True:
16          print("Esperando a recibir datos... ")
17          data = Client_conn.recv(buffer_size)
18          print ("Recibido,", data, " de ", Client_addr)
19  ✓      if not data:
20          break
21          print("Enviando respuesta a", Client_addr)
22          Client_conn.sendall(data)
23

```

Imagen 5.

En la imagen 5, se tienen las líneas de código 13 a 22, en el cuál no se tiene un cierre explícito sin embargo el motivo se encuentra en la declaración “with”, esta palabra reservada es un gestor de contexto que nos ayuda a cerrar la conexión una vez el cliente finaliza su interacción con el servidor, de manera explícita no muestra un “TCPServerSocket.close()” pero realiza la misma función y su intención es evitar dejar el puerto ocupado aun cuando la interacción ya finalizó.

2. Describa la estructura general del programa usando el siguiente esquema:

crear → configurar → _____ → comunicar → cerrar

Crear (socket()) → Configuar (bind()) → Aceptar (listen()/accept()) → Comunicar (recv()/sendall()) → Cerrar (with()/close())

3. Explique por qué accept() genera un nuevo descriptor (conn).

La respuesta se encuentra en que “accept()” genera un nuevo socket por cada cliente que busca conexión con el servidor, esto lo logra con “Client_conn” cuyo propósito es tener un solo socket para un solo cliente.

Parte 2 – Comparación estructural (UDP)

Analice ahora el programa echo_server de la carpeta UDP

Actividades

4. Identifique la línea donde se crea el socket.

`sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)`

5. Compare este programa con el anterior y responda:

- ¿Qué instrucciones ya no aparecen?

En UDP np aparecen estas instrucciones:

`listen()`

`accept()`

`connect()` (del lado del servidor)

El segundo socket (por ejemplo `conn`, `addr = accept()`)

- ¿Por qué UDP no necesita `listen()`?

Porque UDP no es orientado a conexión, el servidor solo espera datagramas y no hay conexión previa.

`data, addr = sock.recvfrom(1024)` --> El servidor hace este comando, que incluye el mensaje y la dirección del cliente.

- ¿Por qué no existe un segundo descriptor como en TCP?

Dado que no existe conexión persistente, sólo existen mensajes independientes.

TCP	UDP
Socket servidor (escucha)	Sólo tiene un socket, el cual recibe y responde
Socket cliente (conexión establecida)	mensajes.

Describe la nueva estructura general del programa.

```
socket(AF_INET, SOCK_DGRAM) //crear el socket UDP
```

```
bind((host, port)) //asociar el socket a una IP y a un puerto
```

```
recvfrom() //esperar mensajes
```

```
sendto() //enviar respuesta al cliente
```

```
while True //para ciclar el proceso
```

En la cuál no hay conexión, no hay aceptación de los clientes y no hay sockets extras

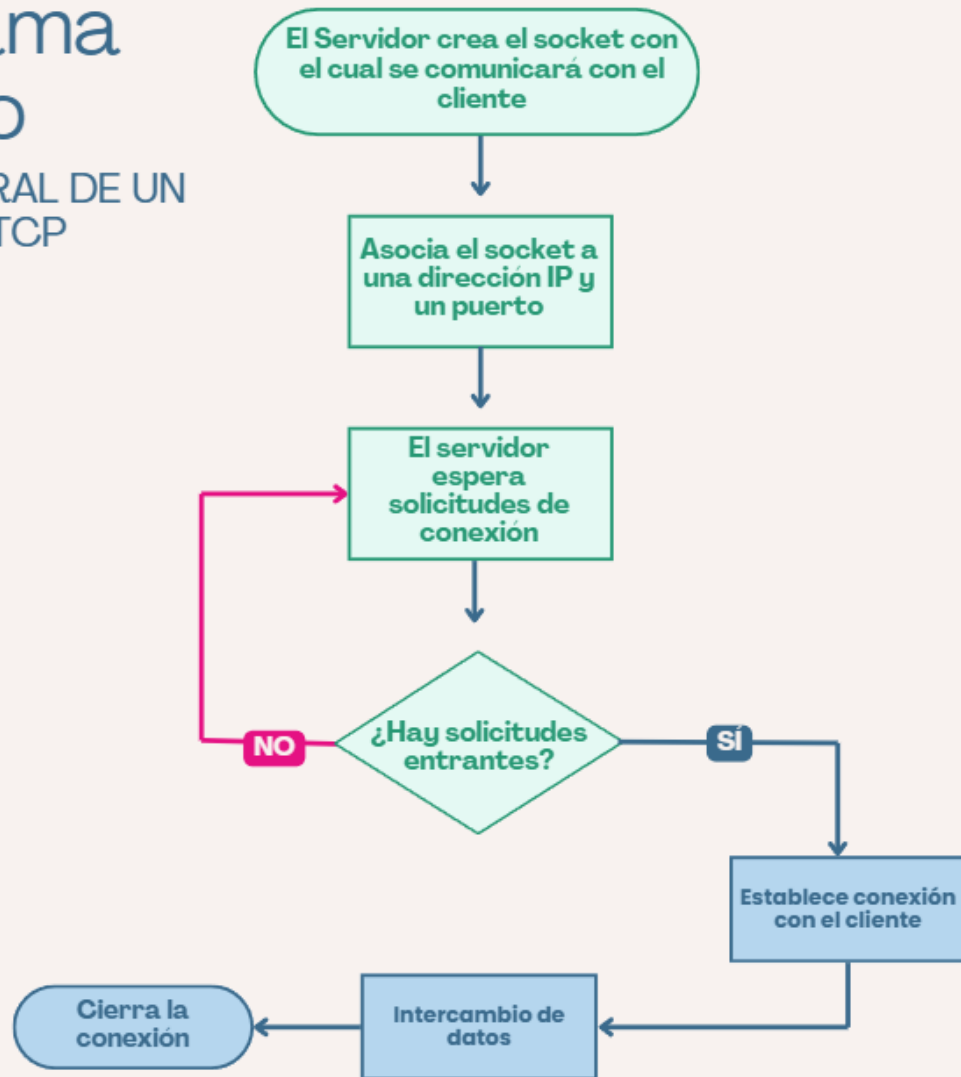
Parte 3 – Diagrama estructural

Construya un diagrama de flujo que represente:

- La estructura del servidor TCP.

Diagrama de flujo

ESTRUCTURAL DE UN SERVIDOR TCP



- La estructura del servidor UDP.

Diagrama de flujo

ESTRUCTURAL DE UN SERVIDOR UDP



Debe mostrarse claramente la diferencia en la fase de establecimiento de conexión.

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(("0.0.0.0", 5000))
data = s.recv(1024)
```

Responda:

6. ¿Qué instrucciones faltan para que funcione correctamente como servidor TCP?

- `socket.AF_INET` // usa IPv4
- `socket.SOCK_STREAM` // nos indica que es TCP
- `bind()` //asocia el socket a una IP y puerto
- `listen()` //pone al servidor en estado de espera de conexiones
- `accept()` //establece la conexión TCP (handshake) y devuelve:
- `conn` // nuevo socket para comunicarte con el cliente
- `addr` // dirección del cliente

No podemos hacer `recv()` directamente después de `bind()` porque todavía no hay conexión establecida.

7. ¿Qué error ocurriría si se ejecuta tal como está?

Transport endpoint isn't connected

El socket está conectado y vinculado al puerto `bind()`, pero al no encontrarse conectado a ningún cliente, entonces la función `recv()` no funciona, únicamente sobre algún socket conectado.

8. ¿Qué cambiaría si se quisiera convertir en un servidor UDP?

Justifique técnicamente sus respuestas.

Es totalmente diferente dado los comandos que utiliza UDP, ya que no utiliza `listen`, `accept` y solo usa `recvfrom`.

`Soc_dgram` que nos indica que estamos trabajando UDP, y no establece conexión

1. Cambiar `SOCK_STREAM` por `SOCK_DGRAM`
2. Usar `recvfrom()` en lugar de `recv()`

En el código UDP:

```
import socket
```

```
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

```
s.bind(("0.0.0.0", 5000))
```

```
data, addr = s.recvfrom(1024)
```

